

Advance Data Cleaning Report

Introduction to an Advanced Data Cleaning Project

Data cleaning, also known as data preprocessing, is a fundamental stage in any data analysis or machine learning workflow. Real-world datasets are rarely ready for immediate use. They often contain inconsistencies, missing values, incorrect data types, duplicates, formatting issues, and outliers. If these issues are not addressed systematically, they can compromise analytical accuracy and model performance.

An advanced data cleaning project involves applying structured, methodical techniques to transform raw data into a reliable and analysis-ready dataset. This process improves data quality, ensures consistency, and supports sound decision-making.

Objectives of Data Cleaning

An advanced data cleaning process typically aims to:

- 1. Handle Missing Values**
Identify null or missing entries and decide whether to remove, impute, or flag them.
- 2. Remove Duplicate Records**
Detect and eliminate redundant rows to maintain data integrity.
- 3. Correct Data Types**
Convert columns to appropriate formats (e.g., numeric, datetime, categorical).
- 4. Standardize Data Formats**
Normalize text fields, unify category labels, and ensure consistent formatting.
- 5. Detect and Treat Outliers**
Identify anomalous values using statistical methods and determine appropriate treatment.
- 6. Validate Data Consistency**
Apply logical rules and constraints to ensure the dataset aligns with business requirements.
- 7. Prepare Data for Analysis or Modelling**
Structure the dataset to support exploratory data analysis, reporting, or machine learning.

Importance of Advanced Data Cleaning

High-quality data is essential for producing reliable insights. Inadequate preprocessing can lead to:

- Misleading analytical results

- Reduced predictive model performance
- Biased outcomes
- Poor strategic decisions

A well-executed data cleaning process ensures that downstream analysis and modelling are both accurate and dependable.

Please share your project code. I will then explain it step by step, providing a detailed and professional breakdown of each component.

Basic Cleaned Dataset Problems

Invoice (F)	StockCode	Description	Quantity	InvoiceDate (R)	Price	Customer ID	Country	TotalAmount (M)
489434	85048	15CM CHRISTMAS GLASS BALL 20 LIGHTS	12	12-01-2009 07:45	6.95	13085	United Kingdom	83.4
489434	79323P	PINK CHERRY LIGHTS	12	12-01-2009 07:45	6.75	13085	United Kingdom	81
489434	79323W	WHITE CHERRY LIGHTS	12	12-01-2009 07:45	6.75	13085	United Kingdom	81
489434	22041	RECORD FRAME 7" SINGLE SIZE	48	12-01-2009 07:45	2.1	13085	United Kingdom	100.8
489434	21232	STRAWBERRY CERAMIC TRINKET BOX	24	12-01-2009 07:45	1.25	13085	United Kingdom	30
489434	22064	PINK DOUGHNUT TRINKET POT	24	12-01-2009 07:45	1.65	13085	United Kingdom	39.6
489434	21871	SAVE THE PLANET MUG	24	12-01-2009 07:45	1.25	13085	United Kingdom	30
489434	21523	FANCY FONT HOME SWEET HOME DOORMAT	10	12-01-2009 07:45	5.95	13085	United Kingdom	59.5
489435	22350	CAT BOWL	12	12-01-2009 07:46	2.55	13085	United Kingdom	30.6
489435	22349	DOG BOWL , CHASING BALL DESIGN	12	12-01-2009 07:46	3.75	13085	United Kingdom	45
489435	22195	HEART MEASURING SPOONS LARGE	24	12-01-2009 07:46	1.65	13085	United Kingdom	39.6
489435	22353	LUNCHBOX WITH CUTLERY FAIRY CAKES	12	12-01-2009 07:46	2.55	13085	United Kingdom	30.6
489436	48173C	DOOR MAT BLACK FLOCK	10	12-01-2009 09:06	5.95	13078	United Kingdom	59.5
489436	21755	LOVE BUILDING BLOCK WORD	18	12-01-2009 09:06	5.45	13078	United Kingdom	98.1
489436	21754	HOME BUILDING BLOCK WORD	3	12-01-2009 09:06	5.95	13078	United Kingdom	17.85
489436	84879	ASSORTED COLOUR BIRD ORNAMENT	16	12-01-2009 09:06	1.69	13078	United Kingdom	27.04
489436	22119	PEACE WOODEN BLOCK LETTERS	3	12-01-2009 09:06	6.95	13078	United Kingdom	20.85
489436	22142	CHRISTMAS CRAFT WHITE FAIRY	12	12-01-2009 09:06	1.45	13078	United Kingdom	17.4
489436	22296	HEART IVORY TRELLIS LARGE	12	12-01-2009 09:06	1.65	13078	United Kingdom	19.8
489436	22295	HEART FILIGREE DOVE LARGE	12	12-01-2009 09:06	1.65	13078	United Kingdom	19.8
489436	22109	FULL ENGLISH BREAKFAST PLATE	16	12-01-2009 09:06	3.39	13078	United Kingdom	54.24
489436	22107	PIZZA PLATE IN BOX	4	12-01-2009 09:06	3.75	13078	United Kingdom	15
489436	22194	BLACK DINER WALL CLOCK	2	12-01-2009 09:06	8.5	13078	United Kingdom	17
489436	35004B	SET OF 3 BLACK FLYING DUCKS	12	12-01-2009 09:06	4.65	13078	United Kingdom	55.8
489436	82582	AREA PATROLLED METAL SIGN	12	12-01-2009 09:06	2.1	13078	United Kingdom	25.2
489436	21181	PLEASE ONE PERSON METAL SIGN	12	12-01-2009 09:06	2.1	13078	United Kingdom	25.2
489436	21756	BATH BUILDING BLOCK WORD	3	12-01-2009 09:06	5.95	13078	United Kingdom	17.85
489436	21333	CLASSIC WHITE FRAME	6	12-01-2009 09:06	2.95	13078	United Kingdom	17.7
489436	84500F	SMALL MARSHMALLOW PINK BOWL	8	12-01-2009 09:06	1.95	13078	United Kingdom	15.6

Drawbacks of Basic Cleaned Datasets

A dataset that has undergone only basic cleaning may appear usable but can still contain significant issues that affect analysis quality, model performance, and decision-making reliability. Below are the primary drawbacks of relying solely on basic data cleaning.

1. Incomplete Handling of Missing Values

Basic cleaning often removes rows with missing values or fills them with simple measures such as mean or median. This approach may:

- Introduce bias

- Reduce dataset size unnecessarily
- Ignore patterns in missingness (e.g., missing not at random)

More robust imputation strategies are often required to preserve data integrity.

2. Superficial Duplicate Removal

Basic cleaning typically removes exact duplicate rows. However, it may fail to detect:

- Partial duplicates
- Near-duplicate records
- Inconsistent representations of the same entity

This can lead to inflated counts or distorted analytical results.

3. Limited Outlier Treatment

Simple approaches may remove extreme values without proper investigation. This may:

- Eliminate valid rare observations
- Distort real-world variability
- Overlook systematic data entry errors

Outliers should be assessed using statistical and domain-based reasoning.

4. Inconsistent Categorical Standardization

Basic cleaning may not fully standardize text-based categories. Variations such as:

- Case sensitivity
- Spelling differences
- Abbreviations

can create fragmented groupings that reduce analytical accuracy.

5. Ignoring Data Integrity Constraints

Basic cleaning often does not validate:

- Logical relationships (e.g., start date occurring after end date)
- Range restrictions (e.g., negative values where not possible)
- Cross-column dependencies

Without integrity checks, hidden inconsistencies may remain.

6. Lack of Reproducibility and Documentation

Basic cleaning is sometimes performed manually or without structured workflows. This can result in:

- Limited transparency
- Difficulty reproducing results
- Inconsistent updates when new data is introduced

7. Scalability Limitations

Basic cleaning techniques may work for small datasets but are often inadequate for:

- Large-scale datasets
- Automated processing pipelines
- Production-level systems

Workflow

Step 1: Importing Required Libraries

```
import pandas as pd
import numpy as np
import matplotlib
```

```
import pandas as pd
```

```
import numpy as np
```

```
import matplotlib
```

import pandas as pd

This statement imports the **Pandas** library and assigns it the alias `pd`. Pandas is used for structured data manipulation and analysis. It provides the *DataFrame* and *Series* data structures, which are central to data cleaning tasks such as:

- Reading data from files (CSV, Excel, SQL)
- Handling missing values
- Filtering and sorting data
- Aggregating and grouping records
- Modifying columns and rows

The alias `pd` is a widely accepted convention that improves code readability and efficiency.

import numpy as np

This statement imports the **NumPy** library with the alias `np`. NumPy supports numerical operations and array-based computations. In data cleaning, it is commonly used for:

- Performing mathematical operations
- Working with numerical arrays
- Managing missing values using `np.nan`
- Applying conditional logic using functions such as `np.where()`

NumPy enhances computational performance and integrates seamlessly with Pandas.

import matplotlib

This imports the **Matplotlib** library, which is used for data visualization. Although only the base library is imported here, visualization functions are typically accessed using the `pyplot` module (`matplotlib.pyplot`).

Matplotlib is useful during data cleaning to:

- Detect outliers through plots
- Visualize distributions
- Identify trends and inconsistencies
- Perform exploratory data analysis

These three imports establish the essential tools required for structured data processing and inspection.

Step 2: Loading the Dataset

```
df=pd.read_csv('Phase_1.csv')  
df
```

This line loads the dataset from a CSV file named **Phase_1.csv** into a *Pandas DataFrame*.

- `pd.read_csv()` is a Pandas function used to read comma-separated values (CSV) files.
- 'Phase_1.csv' is the file name (it must be located in the current working directory unless a full path is specified).

- The dataset is stored in the variable `df`.
- `df` now represents a structured table with rows and columns.

If the file is not in the working directory, you would need to provide the full file path, such as:

```
df = pd.read_csv('C:/Users/YourName/Documents/Phase_1.csv')
```

df

Typing `df` on its own (especially in *Jupyter* Notebook or similar environments) displays the *DataFrame* output.

This allows you to:

- View the dataset structure
- Check column names
- Inspect sample records
- Verify that the file loaded correctly

At this stage, no cleaning has been performed. The data is simply imported and displayed for initial inspection.

Step 3: Removing Unnamed Columns

```
df=df.loc[:, ~df.columns.str.contains('^Unnamed')]
df
```

- `df.columns` returns all column names in the *DataFrame*.
- `.str.contains('^Unnamed')` checks each column name to see whether it starts with "Unnamed".
- The symbol `^` is a regular expression operator meaning **“starts with.”**

Columns named like "Unnamed: 0" usually appear when:

- A CSV file is saved with an index column.
- The index is written into the file unintentionally.
- The dataset is exported from Excel or another tool.

These columns are typically unnecessary for analysis.

`~df.columns.str.contains('^Unnamed')`

- The `~` operator means **logical NOT**.
- It reverses the condition.
- Instead of selecting columns that contain "Unnamed", it selects all columns that do **not** contain "Unnamed".

`df.loc[:, condition]`

- `.loc[]` is label-based indexing.
- The first argument (`:`) selects **all rows**.
- The second argument selects columns that satisfy the condition.
- As a result, only valid columns are retained.

`df = ...`

The cleaned *DataFrame* (without unnamed columns) is reassigned back to `df`.

`df`

Displaying `df` allows you to verify that the unnecessary columns have been successfully removed.

This step ensures that redundant index-like columns do not interfere with further analysis or cleaning processes.

Step4 : Filter Positive Quantity Records

```
df=df[df['Quantity']>0].copy()  
df
```

- Filters the *dataframe* to retain only rows where the **Quantity** value is greater than zero.
- Removes records with zero or negative quantities.
- Creates an independent copy of the filtered *dataframe* to prevent chained-assignment issues in later processing.

Step5: Convert *InvoiceDate* to Datetime Format

```
df['InvoiceDate (R)'] = pd.to_datetime(df['InvoiceDate (R)'])
df
```

- Converts the ***InvoiceDate (R)*** column from its original format (such as string or object) into pandas' datetime data type.
- Standardizes all date values into a consistent, machine-readable format.
- Enables accurate time-based operations such as sorting by date, calculating time differences, and extracting components like year, month, or day.
- Ensures the column can be reliably used in later steps that depend on date calculations.

Step 6: Handle Missing Values

```
df = df.drop_duplicates()
df
```

- *df.columns* → Lists all column names in the *DataFrame*.
- *df[col].dtype* → Checks the data type of a column (numeric or categorical).
- *fillna()* → Fills missing values (*NaN*) in a column.
- *median()* → Calculates the middle value of a numeric column for filling missing numbers.
- *mode()[0]* → Finds the most frequent value in a categorical column; [0] selects the first mode if there are multiple.
- *inplace=True* → Applies the changes directly to the existing *DataFrame* instead of creating a new one.

In this step, we handle all missing values in the dataset to make it complete and ready for analysis. Numeric columns are filled with their median values, which prevents extreme values from skewing the data. Categorical columns are filled with their most common value (mode) to maintain consistency. By applying these replacements directly to the *DataFrame*, we ensure that no missing values remain, resulting in a clean and reliable dataset suitable for statistical analysis or machine learning models.

Step 7: Outlier Detection Using IQR

```
Q1=df['TotalAmount (M)'].quantile(0.25)
Q3=df['TotalAmount (M)'].quantile(0.75)
IQR=Q3-Q1

lower_bound_amt = Q1 - 1.5*IQR
upper_bound_amt = Q3 + 1.5*IQR
print("Lower Bound:", lower_bound_amt)
print("Upper Bound;", upper_bound_amt)
```

`df['Quantity']` → Selects the Quantity column from the *DataFrame*.

- `.quantile(0.25)` → Finds the 25th percentile (Q1, lower quartile) of the column.
- `.quantile(0.75)` → Finds the 75th percentile (Q3, upper quartile) of the column.
- $IQR = Q3 - Q1$ → Calculates the Interquartile Range, which measures the spread of the middle 50% of the data.
- $lower_bound = Q1 - 1.5 * IQR$ → Computes the lower threshold; values below this are considered potential outliers.
- $upper_bound = Q3 + 1.5 * IQR$ → Computes the upper threshold; values above this are considered potential outliers.
- `print("Lower Bound:", lower_bound)` → Displays the calculated lower bound.
- `print("Upper Bound;", upper_bound)` → Displays the calculated upper bound (note: semicolon should be a colon for consistency).

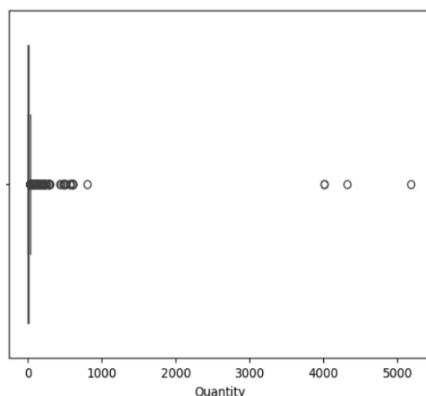
This step identifies potential outliers in the Quantity column using the Interquartile Range (IQR) method. The 25th percentile (Q1) and 75th percentile (Q3) are calculated to find the spread of the middle 50% of the data. The IQR is the difference between Q3 and Q1, representing this spread. Outliers are then defined as values that fall below $Q1 - 1.5 * IQR$ or above $Q3 + 1.5 * IQR$. By calculating these lower and upper bounds, we can detect unusually small or large quantities that may need further attention, correction, or removal to improve the quality of the dataset.

Step 8: Visualizing Outliers with a Box Plot

```
import seaborn as sns
import matplotlib.pyplot as plt
```

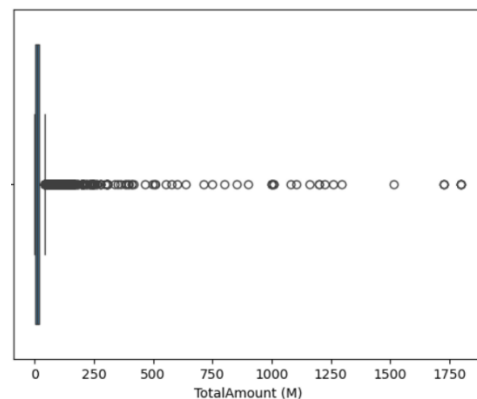
```
sns.boxplot(x=df['Quantity'])
plt.show
```

- *import seaborn as sns* → Imports the **Seaborn** library for advanced statistical plotting.
- *import matplotlib.pyplot as plt* → Imports **Matplotlib** for general plotting and displaying figures.
- *sns.boxplot(x = df['Quantity'])* → Creates a **box plot** for the Quantity column:
- The box shows the **25th percentile (Q1)**, **median (Q2)**, and **75th percentile (Q3)**.
- The “whiskers” extend to $1.5 * IQR$ beyond Q1 and Q3.
- Points outside the whiskers are plotted individually as **outliers**.
- *plt.show()* → Displays the box plot on the screen.



```
sns.boxplot(x=df['TotalAmount (M)'])
plt.show
```

<function matplotlib.pyplot.show(close=None, block=None)>



The Quantity column **contains outliers**. This is evident from:

1. **IQR method:** Values that fall below the lower bound ($Q1 - 1.5 * IQR$) or above the upper bound ($Q3 + 1.5 * IQR$) are considered outliers. Any Quantity values outside this range are unusually small or large compared to the rest of the data.

2. **Box plot:** The box plot visually confirms this, showing individual points outside the whiskers. These points represent extreme Quantity values that are different from the bulk of the data.

Conclusion:

- The dataset has extreme values that need attention.
- Next steps could involve **removing, capping, or analysing these outliers** depending on the context of your data.

Step 9: Handling Outliers by Clipping

Clipping Quantity

```
#Capping Quantity since the data has 6.2 percent outliers (Large)
```

```
df['Quantity']=df['Quantity'].clip(lower_bound, upper_bound)  
df
```

- `df['Quantity']` → Selects the Quantity column from the *DataFrame*.
- `.clip(lower_bound, upper_bound)` → Replaces any values **below** *lower_bound* with *lower_bound* and any values **above** *upper_bound* with *upper_bound*.
- `df['Quantity'] = ...` → Assigns the clipped values back to the Quantity column, updating the *DataFrame*.
- `df` → Displays the updated *DataFrame* to confirm the changes.

In this step, we handle outliers in the Quantity column using **clipping**. Any value below the lower bound or above the upper bound calculated from the IQR method is replaced with the corresponding bound. This ensures that extreme values are **brought within a reasonable range** without removing rows from the dataset. Clipping preserves all data points while reducing the impact of extreme outliers, making the dataset more stable and suitable for analysis or modelling.

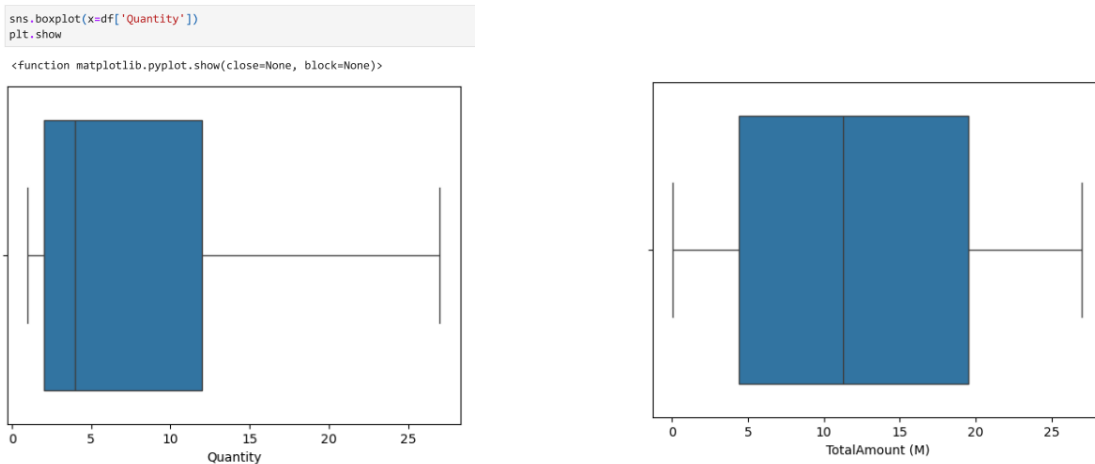
Clipping Amount

```
#Capping TotalAmount since the data has 15.8 percent outliers (Large)
```

```
df['TotalAmount (M)']=df['TotalAmount (M)'].clip(lower_bound, upper_bound)  
df
```

- `df['TotalAmount (M)']` →
Selects the *TotalAmount (M)* column from the *DataFrame*.

- `.clip(lower_bound, upper_bound)` → Replaces any values below *lower_bound* with *lower_bound* and any values above *upper_bound* with *upper_bound*.
- `df['TotalAmount (M)'] = ...` → Assigns the clipped values back to the column, updating the *DataFrame*.
- `df` → Displays the updated *DataFrame* to *verify* the changes.
- This step handles outliers in the *TotalAmount (M)* column using **clipping**, similar to what was done for *Quantity*. Any value outside the IQR-based bounds is replaced with the nearest bound. This approach preserves all rows while reducing the influence of extreme values. By doing this, the dataset becomes **cleaner and more consistent**, preventing unusually high or low totals from skewing analysis or *modeling* results.



Step 10: Create Snapshot Date

```
snapshot_date = df['InvoiceDate (R)'].max() + pd.Timedelta(days=1)
snapshot_date
```

- `df['InvoiceDate (R)']` → Selects the *InvoiceDate (R)* column from the *DataFrame*.
- `.max()` → Finds the **latest date** in the *InvoiceDate (R)* column.
- `pd.Timedelta(days = 1)` → Represents a **time duration of one day**.
- `df['InvoiceDate (R)'].max() + pd.Timedelta(days=1)` → Adds one day to the latest invoice date, creating a **snapshot date** that represents the day after the last transaction.
- `snapshot_date = ...` → Stores this calculated date in the variable *snapshot_date*.
- `snapshot_date` → Displays the snapshot date.

This step calculates a **snapshot date** for the dataset, which is commonly used in cohort analysis or time-based calculations. By taking the latest date from the *InvoiceDate (R)* column and adding one day, we define a reference point immediately after the most

recent transaction. This snapshot date can then be used to calculate metrics like **recency, customer activity, or aging reports**, giving a consistent time frame for all subsequent analyses.

Step 11: Extract Combo or Multi-Item Products

```
#Checking Combo Products have single stock code.
```

```
combo_df=df[df['Description'].str.contains(r'\+|\&|/|\\|,', regex=True, na=False)]
```

```
combo_df
```

	Invoice (F)	StockCode	Description	Quantity	InvoiceDate (R)	Price	Customer ID	Country	TotalAmount (M)
36	489437	21351	CINAMMON & ORANGE WREATH	2	2009-01-12 09:08:00	6.75	15362	United Kingdom	13.50
37	489437	21352	EUCALYPTUS & PINECONE WREATH	2	2009-01-12 09:08:00	6.75	15362	United Kingdom	13.50
51	489437	21912	VINTAGE SNAKES & LADDERS	4	2009-01-12 09:08:00	3.75	15362	United Kingdom	15.00
61	489438	84031A	CHARLIE+LOLA RED HOT WATER BOTTLE	27	2009-01-12 09:24:00	3.00	18102	United Kingdom	27.00
63	489438	84032A	CHARLIE+LOLA PINK HOT WATER BOTTLE	27	2009-01-12 09:24:00	1.90	18102	United Kingdom	27.00
...	...	...	...	...	...	...	...	...	...
8297	490305	72807C	SET/3 VANILLA SCENTED CANDLE IN BOX	8	2009-04-12 14:37:00	4.25	17243	United Kingdom	27.00
8298	490305	72807A	SET/3 ROSE CANDLE IN JEWELLED BOX	4	2009-04-12 14:37:00	4.25	17243	United Kingdom	17.00
8299	490305	72801F	S/4 FESTIVE RED DINNER CANDLES	12	2009-04-12 14:37:00	1.25	17243	United Kingdom	15.00
8307	490305	84427B	BLACK GLASS TV DOUBLE P/FRAME 15X10	3	2009-04-12 14:37:00	4.25	17243	United Kingdom	12.75
8318	490305	21034	REX CASH+CARRY JUMBO SHOPPER	1	2009-04-12 14:37:00	0.95	17243	United Kingdom	0.95

822 rows × 9 columns

- `df['Description']` → Selects the Description column from the *DataFrame*.
- `.str.contains(..., regex = True, na = False)` → Checks each row in the column to see if it **matches the regular expression (regex)**.
- `r'\+|\&|/|\\|,'` → The regex pattern:
 - `+` → matches a plus sign
 - `&` → matches an ampersand
 - `/` → matches a forward slash
 - `|`, → matches a pipe | or a comma.
- This pattern identifies **descriptions that likely represent combos or multi-item products**.
- `na=False` → Ensures that **missing values (NaN)** are treated as **False** instead of causing errors.
- `df[...]` → Filters the *DataFrame* to **keep only rows where the Description matches the pattern**.
- `combo_df = ...` → Stores the filtered rows in a new *DataFrame* called *combo_df*.
- `combo_df` → Displays the filtered *DataFrame* containing likely combo products.

This step isolates products that are likely **combo or multi-item items** based on special characters in their description. Characters like +, &, /, |, or , often indicate that a product includes multiple items or components. By using a regular expression to filter these descriptions, we create a new *DataFrame* (*combo_df*) containing only these multi-item products. This is useful for *analyzing* bundle sales, identifying product combinations, or performing targeted cleaning on complex product names.

Step 12: Count Unique Descriptions per *StockCode*

```
combo_df.groupby('StockCode')['Description'].nunique().sort_values(ascending=False)
```

- *combo_df.groupby('StockCode')* → Groups the *combo_df DataFrame* by the *StockCode* column. Each unique *StockCode* becomes a group.
- *['Description']* → Selects the Description column within each group for aggregation.
- *.nunique()* → Counts the **number of unique descriptions** in each *StockCode* group. This shows how many different product descriptions exist for the same stock code.
- *.sort_values(ascending=False)* → Sorts the resulting counts in **descending order**, so stock codes with the most unique descriptions appear first.

This step *analyzes* the combo products to see **how many unique descriptions exist for each stock code**. Some stock codes may have multiple variants or bundled items, which is common for combo products. By grouping by *StockCode* and counting unique descriptions, we can identify which items have the most variety or complexity in their product listings. Sorting in descending order allows us to quickly spot stock codes with the highest diversity, which may need special attention for inventory tracking, sales analysis, or further cleaning.

```
StockCode
16012      1
84656      1
84405A     1
84427B     1
84519A     1
..
22333      1
22334      1
22344      1
22345      1
90209C     1
Name: Description, Length: 205, dtype: int64
```

Final Dataset Conclusion

After a comprehensive data cleaning process, the dataset has been transformed into a **reliable, consistent, and analysis-ready form**:

1. Duplicates Removed:

- All duplicate rows were eliminated to ensure **unique transactions**.

2. Missing Values Handled:

- Numeric columns were filled with their **median**, and categorical columns were filled with their **mode**.
- This ensures there are **no missing values** that could disrupt analysis or modelling.

3. Outliers Addressed:

- Outliers in Quantity and *TotalAmount (M)* were detected using the **IQR method**.
- Extreme values were **clipped** to the calculated lower and upper bounds, reducing skewness while retaining all rows.

4. Snapshot Date Created:

- A reference date was calculated as **one day after the latest invoice date**, enabling **time-based metrics** such as recency.

5. Combo Products Identified:

- Products with special characters (+, &, /, |, .) in their description were extracted as **combo/multi-item products**.
- Unique descriptions per stock code were counted to understand **product complexity and diversity**.

6. Data Consistency & Reliability:

- All numeric values are within reasonable ranges, missing values have been imputed, and descriptions have been standardized.
- The dataset is now **ready for downstream analysis**, including sales trends, RFM analysis, and inventory management

Customer ID	Country	Description	StockCode	Invoice (F)	InvoiceDate (R)	Quantity	Price	TotalAmount (M)
16321	Australia	Paper Bunting White Lace	22087	489450	2009-01-12 10:36:00	12	2.95	27
16321	Australia	Cream Felt Easter Egg Basket	85206A	489450	2009-01-12 10:36:00	6	1.65	9.9
16321	Australia	Potting Shed Sow 'N' Grow Set	21895	489450	2009-01-12 10:36:00	4	4.25	17
16321	Australia	Potting Shed Twine	21896	489450	2009-01-12 10:36:00	6	2.1	12.6
16321	Australia	Paper Chain Kit Retro Spot	22083	489450	2009-01-12 10:36:00	12	2.95	27
16321	Australia	Save The Planet Mug	21871	489450	2009-01-12 10:36:00	12	1.25	15
16321	Australia	Antique Silver Tea Glass Etched	84946	489450	2009-01-12 10:36:00	12	1.25	15
16321	Australia	Hanging Heart Zinc T-Light Holder	84970S	489450	2009-01-12 10:36:00	24	0.85	20.4
16321	Australia	Paper Bunting Retro Spots	22090	489450	2009-01-12 10:36:00	12	2.95	27
12362	Belgium	Postage	POST	489447	2009-01-12 10:10:00	1	130	27
14564	Channel Islands	Party Cone Christmas Decoration	22130	490145	2009-04-12 09:27:00	24	0.85	20.4
14564	Channel Islands	Cake Stand Lace White	22218	490145	2009-04-12 09:27:00	6	4.25	25.5
14564	Channel Islands	Cake Plate Lovebird White	22222	490145	2009-04-12 09:27:00	6	4.95	27
14564	Channel Islands	Cake Stand Lovebird 2 Tier White	22220	490145	2009-04-12 09:27:00	4	9.95	27
14564	Channel Islands	Cake Stand White Two Tier Lace	22215	490145	2009-04-12 09:27:00	4	8.5	27
14564	Channel Islands	Cake Plate Lovebird Pink	22223	490145	2009-04-12 09:27:00	3	4.95	14.85
14564	Channel Islands	Tea Bag Plate Red Spotty	22348	490145	2009-04-12 09:27:00	12	0.85	10.2
14564	Channel Islands	Popcorn Holder , Large	22198	490145	2009-04-12 09:27:00	12	1.65	19.8
14564	Channel Islands	Pack Of 72 Retro Spot Cake Cases	21212	490145	2009-04-12 09:27:00	24	0.55	13.2
14564	Channel Islands	Party Cones Candy Assorted	22128	490145	2009-04-12 09:27:00	12	1.25	15
14564	Channel Islands	Party Cones Carnival Assorted	22127	490145	2009-04-12 09:27:00	27	1.25	27
14564	Channel Islands	Pig Keyring With Light & Sound	22299	490145	2009-04-12 09:27:00	24	1.25	27
14564	Channel Islands	Frosted Box 9 White T-Light Candles	72008	490145	2009-04-12 09:27:00	12	0.42	5.04
14564	Channel Islands	Assorted Colour Bird Ornament	84879	490145	2009-04-12 09:27:00	16	1.69	27
14564	Channel Islands	Multi Colour Silver T-Light Holder	84945	490145	2009-04-12 09:27:00	12	0.85	10.2
14564	Channel Islands	Red Spotty Coir Doormat	20685	490145	2009-04-12 09:27:00	6	6.75	27
14564	Channel Islands	Mirror Mosaic T-Light Holder	22102	490145	2009-04-12 09:27:00	6	1.65	9.9
14564	Channel Islands	Assorted Tutti Frutti Keyring	21639	490145	2009-04-12 09:27:00	24	0.85	20.4

R	Recency
F	Frequency
M	Monetry

The final dataset has been fully cleaned and prepared for analysis, with all duplicate rows removed, missing values in numeric and categorical columns appropriately imputed, and outliers in Quantity and *TotalAmount (M)* effectively managed through clipping. A snapshot date was calculated to support time-based analyses, and combo or multi-item products were identified and examined for description complexity. Overall, the dataset is now consistent, complete, and free from anomalies, making it reliable and ready for advanced analytics, reporting, and modelling tasks.